Department of Computer Science and Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2: PROBLEM STATEMENTS

CO2 coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search. **Nature:** Individual. **Duration:** 1 week. **Total marks:** 50.

Q1 – Sorting Under Different Data Conditions (6 Marks)

Context – E-commerce order processing. An e-commerce fulfilment centre arranges customer orders according to a dispatch priority score. The team is considering Selection Sort, Bubble Sort and Insertion Sort, but incoming data may be randomly ordered, nearly sorted, already sorted or significantly disordered.

Tasks

- Investigate whether one algorithm should always be preferred or whether the choice depends on the characteristics of the input data.
- Before implementation, predict how the three algorithms behave under different input arrangements and explain the algorithmic reason.
- Design two deliberately contrasting datasets and one additional edge-case dataset that expose behavioural differences.
- Implement the three algorithms and record comparisons, swaps/movements and (where useful) execution time; compare with your prediction.
- **Design decision:** about 95% of existing orders are already in the correct priority order, with only small batches of new orders added periodically. Recommend and defend one algorithm.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? (4 Marks)

Context – Aircraft maintenance tool tracking. A maintenance facility keeps a temporary list of specialised tools issued to technicians. Each record has Tool ID, Technician ID, Issue Time and Work Bay. The list changes continuously. When a technician asks “Who currently has Tool T-184?”, the system uses Sequential Search.

Tasks

- Design experiments for successful searches at different positions, unsuccessful searches and different list sizes; record the number of comparisons.
- Answer: under what operating conditions is Sequential Search reasonable, and when would you stop recommending it? Consider dataset size and frequency of search requests.
- **Changed condition:** the organisation expands from one hangar to 20 maintenance facilities, with substantially more active records and lookup requests. Re-evaluate your recommendation.

Q3 – Engineer the Behaviour of Brute-Force String Matching (6 Marks)

Context – Cybersecurity log screening. A security operations centre searches large text-based system logs for known event signatures represented as character patterns. The initial system uses Brute-Force String Matching.

Tasks

- Design (1) a text-pattern pair where the algorithm does relatively little work, (2) a pair engineered to cause a large number of character comparisons, and (3) one additional test condition of your own.
- Before each test, predict the behaviour and explain the algorithmic reason. Implement the algorithm and record character comparisons.
- Compare prediction with result and explain why one input needs much more work than another, which text and pattern characteristics create repeated comparisons, and what this reveals about evaluating an algorithm using only input size.

Q4 – Closest Pair and the Cost of Examining Every Possibility (6 Marks)

Context – Drone collision-risk monitoring. A logistics company operates delivery drones. Each drone reports its position Di = (xi, yi). The monitoring system must identify the two drones closest to one another.

Tasks

- Design a Brute-Force Closest-Pair solution: determine which candidate pairs must be examined, how duplicate pair comparisons are avoided, and how many unique pairs exist for n drones (derive mathematically).
- Implement and test with at least three meaningfully different input sizes; record pairs examined, distance calculations and (where useful) execution time; compare theoretical growth with experiment.
- **Scalability inquiry:** define an acceptable response time, estimate when brute force becomes impractical, identify where the inefficiency originates and what a redesigned strategy would need to avoid or reduce. An advanced algorithm need not be implemented.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle (7 Marks)

Context – Wildlife reserve boundary monitoring. Monitoring stations with coordinates Pi = (xi, yi) are spread across a reserve. Officers need the outer boundary formed by the stations. Develop a Brute-Force Convex Hull solution from only this principle: a line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.

Tasks

- Convert the principle into a procedure: how two points define a line, how another point's position relative to the line is determined, how remaining points are examined, what condition identifies a hull edge, and how the final boundary is constructed. Write your own pseudocode first.
- Test with a point set containing boundary and interior points, and a second configuration designed by you to challenge the algorithm. Visualise the boundary and analyse how computational work grows with the number of points.
- **Correctness inquiry:** explain, using geometric reasoning rather than program output, why your test shows an accepted edge belongs to the outer boundary.

Q6 – Exhaustive Search and Combinatorial Growth (8 Marks)

Context – Humanitarian airlift planning. A helicopter carrying emergency supplies has a strict payload limit. Each package has Package ID, Weight and Humanitarian Value. Select a combination of packages that does not exceed the payload limit and maximises total humanitarian value.

Tasks

- Design an Exhaustive-Search strategy: how combinations are represented, how candidates are generated, how feasibility is checked, how feasible solutions are evaluated, and how the best is retained (without supplied pseudocode).
- **Correctness check:** build a small instance whose combinations can be verified independently and show the algorithm finds the correct optimum.
- **Growth investigation:** before experimenting, derive/predict how the number of candidate combinations grows with n; validate with at least three meaningful problem sizes.
- **Inquiry:** explain why a small number of additional packages can cause a dramatic increase in effort, relating it to the growth of the solution space.
- **Deployment decision:** choose and justify a fixed response time and estimate the largest problem size for which you would deploy the exhaustive-search solution.

Q7 – Diagnose, Redesign and Validate (Day-4 Design Challenge) (13 Marks)

Review the evidence from Q1–Q6 and select ONE solution that shows a significant computational limitation as problem size or operating requirement grows. You may not simply replace it with a more advanced textbook algorithm and state that its complexity is better; understand the problem first, then develop an appropriate redesign.

Tasks

- **A. Diagnose the bottleneck (2 marks):** where most effort is spent, which operation grows, whether work is repeated, and which characteristic of the original algorithm causes the limitation, supported by your own evidence.
- **B. Define the design objective (2 marks):** state what the redesign must achieve, with a measurable objective where possible.
- **C. Design the revised strategy (4 marks):** explain what changed, what property is exploited, which operations are avoided, whether preprocessing or extra memory is needed, and new trade-offs; give pseudocode or a design representation.
- **D. Predict before implementing (1 mark):** predict how behaviour should differ and which performance measure should improve.
- **E. Implement and validate (2 marks):** compare original and redesigned solutions on the same or equivalent tests using comparisons, key operations, candidates examined, distance calculations, character comparisons or time.
- **F. Defend the final design (2 marks):** evaluate whether the objective was achieved, whether the prediction held, magnitude of improvement, new costs, when the original may still be preferable, and whether the limitation is sufficiently addressed.

Submission Requirements (All questions)

- Assignment report with the required evidence for Q1–Q7.
- Source code for all implemented solutions.
- Raw experimental data used for the results.
- Tables and graphs used in the analysis, where applicable.
- Operation counts and algorithm-specific measures are the primary evidence; execution time is additional evidence and must not be the sole basis for conclusions.